

《面向对象程序设计实践》

课程论文

题目：	《思维导图绘制工具》
专业：	软件工程
班级：	2024 级 5 班
小组成员 1：	202425220521 阮贵鹏
2：	202425220522 韦成东
指导老师：	彭红星
提交时间：	2026 年 5 月 17 日

目录

- 1 系统分析..... 4
 - 1.1 问题描述 4
 - 1.2 系统功能分析..... 4
 - 1.3 开发平台及工具介绍 4
- 2 系统设计..... 4
 - 2.1 系统总体结构设计 4
 - 2.2 系统各个类及类间关系设计..... 5
 - 2.2.1 入口类..... 5
 - 2.2.2 控制类..... 6
 - 2.2.3 模型层..... 6
 - 2.2.4 命令层..... 7
 - 2.2.5 视图层..... 7
 - 2.2.6 工具层..... 8
 - 2.3 数据存储的设计 8
 - 2.3.1 文件导入/导出 8
 - 2.3.2 内存数据结构..... 8
 - 2.4 界面设计 8
 - 2.4.1 工具栏..... 8
 - 2.4.2 画布区..... 9
- 3 系统实现..... 9
 - 3.1 应用启动流程 9
 - 3.2 命令模式与撤销/重做机制 10
 - 3.3 布局算法实现..... 10
 - 3.4 画布绘制、坐标变换 12
 - 3.5 文件读写实现..... 13
 - 3.6 用户交互实现..... 14
- 4 系统测试..... 14
 - 4.1 模块测试 14
 - 4.1.1 节点添加功能..... 15
 - 4.1.2 节点删除与撤销 15

4.1.3	文本编辑	16
4.1.4	剪贴板操作	18
4.1.5	布局模式切换	20
4.1.6	文件保存与加载	21
4.1.7	导出图片	22
5	系统界面	23
5.1	欢迎界面	23
5.2	主编辑器界面	24
6	总结	24

1 系统分析

1.1 问题描述

思维导图是一种用中央关键词或想法引起形象化构造和分类的图解方式,通过辐射线形连接所有的代表字词、想法或任务。本项目旨在开发一款基于 Java Swing 的思维导图绘制工具,实现思维导图的创建、编辑、存储、导出及多种布局展示功能,帮助用户以树形结构可视化管理思维与信息。

1.2 系统功能分析

根据需求,构建系统主要包含以下功能:

1) 新建,新建一个思维导图。新的思维导图初始时只有一个“中心节点”,在绘图区域展示该思维导图。

2) 保存,将当前正在编辑的思维导图保存到文件中,首次保存一个思维导图时,需要提示用户输入文件名。思维导图文件的扩展名可以自行定义,如 .dt。

3) 打开,从磁盘中选择一个已经存在的思维导图文件,将思维导图显示在绘图区,其对应的树形结构显示在结构显示区。

4) 导出,将当前正在编辑的思维导图导出为 jpg 和 png 格式的图像文件。

5) 选中节点,使用鼠标点击思维导图中的某个节点时,该节点处于选中状态。处于选中状态节点的外观应该与其他节点进行区别,例如使用不同的背景色或边框色。

当选中一个节点时,其他节点均处于未选中状态(如果考虑多选,请自行设计)。鼠标点击不在任何节点中时,所有节点均取消选中状态。

6) 添加节点,使用本功能前,应保证有一个节点为选中状态。

- 如果选中状态的是“中心节点”,则为可以为选中节点添加“子节点”;
- 如果选中状态的是“其他节点”,则为可以为选中节点添加“子节点”,也可以为选中节点添加“兄弟节点”功能,兄弟节点是指选中节点的父节点的子节点。

节点添加后,“结构显示区”同步更新思维导图的结构。

7) 删除节点,使用本功能前,应保证有一个节点为选中状态。删除一个节点时,应该同时删除该节点所有的子节点。节点删除后,“结构显示区”同步更新思维导图的结构。

8) 设置布局,当选中“中心节点”时,可以设置布局方式。程序需要支持 3 种节点的布局排列方式:

- 自动布局,中心节点的所有子节点平均排列在其左、右两侧,两侧节点的排列具有相近的高度(包括所有子节点)。
- 左侧布局,中心节点的所有子节点排列在其左侧。
- 右侧布局,中心节点的所有子节点排列在其右侧。

1.3 开发平台及工具介绍

KidxMind 是一个基于 JavaFX 17 的思维导图绘制工具,采用 MVC 架构。IntelliJ IDEA 浅色主题风格,Material Design 2 图标,用 JDK 17 编译。

2 系统设计

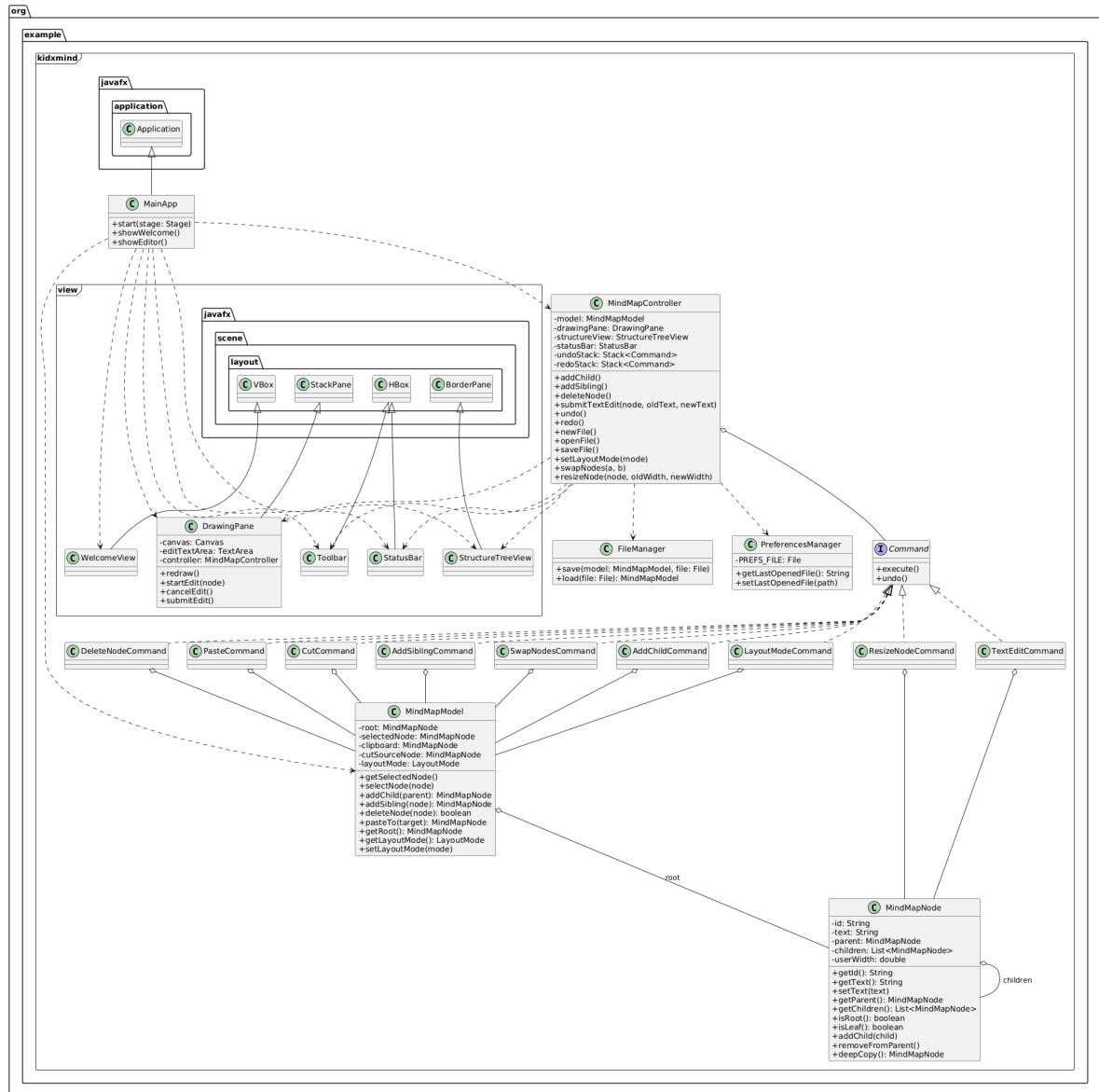
2.1 系统总体结构设计

系统采用 MVC (Model-View-Controller) 设计模式:

- **Model (模型):** 负责维护思维导图的树形数据结构、节点属性及父子关系。
- **View (视图):** 分为绘图区(自定义 JPanel 绘制连线与节点)和结构展示区(JTree 组件),负责数据可视化。
- **Controller (控制器):** 处理用户鼠标事件及工具栏按钮事件,调用模型更新数据,

并通知视图刷新。

2.2 系统各个类及类间关系设计



此为整个项目的UML类图，依此分门介绍各类。

2.2.1 入口类

MainApp 继承自 javafx.application.Application，是 JavaFX 应用程序的标准入口。其核心职责是：

- 1.生命周期管理：在 start(Stage)方法中完成 MVC 三层的初始化——先创建 MindMapModel（Model），再创建 MindMapController（Controller），最后创建各 View 组件。
- 2.启动流程控制：通过 PreferencesManager 读取用户偏好文件，若存在历史文件则直接加载进入编辑器（showEditor()），否则显示欢迎页（showWelcome()）。
- 3.界面组装：使用 AnchorPane+VBox+SplitPane 的嵌套布局，将 Toolbar、DrawingPane、StructureTreeView、StatusBar 组装为完整的编辑器界面。
- 4.依赖注入：通过构造器或 setter 方法将 Controller 注入各 View 组件，同时将 View 组件反向注入 Controller，建立双向关联。

类间关系：MainApp 与 MindMapController 是创建与持有关系；与各 View 组件是组装关系；

与 PreferencesManager 是调用关系。

2.2.2 控制类

MindMapController 是系统的核心调度者，在 MVC 架构中扮演 Controller 角色。其核心职责包括：

- 1.操作分发：接收来自 UI 层（工具栏按钮、键盘快捷键、鼠标事件）的操作请求，将其转化为具体的命令对象并执行。例如 addChildNode()方法创建 AddChildCommand 并调用 executeCommand()。
- 2.命令历史管理：维护两个 Stack<Command>——undoStack（撤回栈）和 redoStack（重做栈）。executeCommand()方法将命令推入 undoStack 并清空 redoStack；undo()从 undoStack 弹出命令并推入 redoStack；redo()反之。
- 3.界面刷新调度：每次命令执行后调用 refreshAll()，依次刷新 DrawingPane.redraw()、StructureTreeView.refresh()和 StatusBar.update()。
- 4.前置条件检查：在执行操作前验证条件（如"删除节点时不能删除根节点"、"粘贴时剪贴板不能为空"）。
- 5.文件操作：封装 FileManager 的静态方法，提供 newFile()、openFile()、saveFile()等文件操作，并在操作前后处理脏标记检查和偏好记录更新。

类间关系：

- 持有 MindMapModel 引用（聚合关系），所有数据操作通过 Model 完成
- 持有 DrawingPane、StructureTreeView、StatusBar 引用（关联关系），通过 setter 注入
- 创建并持有 Command 对象（依赖关系），通过 executeCommand()统一管理
- 调用 FileManager 和 PreferencesManager 的静态方法（依赖关系）

2.2.3 模型层

1) MindMapModel 类

MindMapModel 是系统的数据核心，管理思维导图的全部数据状态，不涉及任何 UI 渲染。其核心职责包括：

- 1.树结构管理：持有根节点 root（MindMapNode 类型），整棵树的层次结构由节点自身的 parent/children 引用维护。
- 2.选中状态管理：维护 selectedNode 字段（单选模式，最多一个节点被选中），提供 selectNode()、clearSelection()、isSelected()方法。
- 3.剪贴板管理：维护 clipboard（深拷贝的子树副本）、clipboardCut（剪切/复制标记）、cutSourceNode（剪切源节点引用）。剪切操作采用"延迟删除"策略——剪切时不立即删除节点，而是在粘贴时由 pasteTo()方法删除源节点。
- 4.布局模式管理：维护 LayoutMode 枚举（AUTO/LEFT/RIGHT），控制子节点在根节点左右两侧的分布方式。
- 5.脏标记管理：维护 dirty 布尔值，标记自上次保存后是否有未保存的修改。

类间关系：

- 持有 MindMapNode 引用（组合关系，Model 拥有根节点，根节点拥有子树）
- 包含 LayoutMode 枚举（内部定义）

2) MindMapNode 类

MindMapNode 是思维导图的核心数据结构，每个节点是树中的一个结点。其核心属性包括：

- 1.基本属性：id（UUID 唯一标识）、text（显示文本）
- 2.树形关系：parent（父节点引用，根节点为 null）、children（子节点列表，List<MindMapNode> 类型）
- 3.布局属性：x、y（节点中心坐标）、width、height（节点尺寸）、userWidth（用户自定义宽

度)、leftSide (左右侧标记)

4.关键方法:

- addChild()/addChildAt(): 添加子节点并建立双向关联 (设置 child.parent=this)
- removeChild()/removeFromParent(): 断开父子关系
- deepCopy(): 递归深拷贝整棵子树, 用于剪贴板操作
- getConnectionRightX()/getConnectionLeftX(): 计算连线出口/入口坐标

类间关系:

- 与自身形成自关联 (parent 和 children 均指向 MindMapNode), 构成树形结构
- 被 MindMapModel 组合持有

2.2.4 命令层

1) Command 接口

定义三个方法签名:

- execute() : 执行操作
- undo() : 撤销操作
- getDescription() : 返回操作的中文描述

Controller 只与 Command 接口交互, 不依赖具体命令类, 体现了 面向接口编程 和 多态 原则。

另九个命令实现类实现 command 接口:

1. **AddChildCommand**
2. **AddSiblingCommand**
3. **DeleteNodeCommand**
4. **TextEditCommand**
5. **CutCommand**
6. **PasteCommand**
7. **SwapNodesCommand**
8. **ResizeNodeCommand**
9. **LayoutModeCommand**

所有命令类持有 MindMapModel 或 MindMapNode 引用, Controller 创建并使用命令对象。

2.2.5 视图层

1) DrawingPane 类

继承 javafx.scene.layout.StackPane, 是系统的核心渲染组件。职责包括:

- 1.Canvas 绘制: 使用 GraphicsContext 在 Canvas 上绘制节点矩形 (fillRoundRect/strokeRoundRect)、连线 (贝塞尔曲线 quadraticCurveTo) 和文字。
- 2.布局算法: 实现三种布局模式——AUTO (自动平衡, 遍历分割点使左右高度最接近)、LEFT (全部左侧)、RIGHT (全部右侧)。采用递归的深度优先遍历计算每个节点的坐标。
- 3.交互处理: 处理鼠标点击 (选中节点)、双击 (开始编辑)、拖拽 (交换节点/调整宽度)、滚轮缩放等事件。
- 4.文本编辑: 管理 TextArea 编辑浮层, 支持实时文本更新和 Enter 键提交。
- 5.坐标变换: 通过三层变换 (平移→缩放→居中) 将逻辑坐标映射到屏幕坐标。

类间关系: 持有 MindMapController 引用 (关联关系), 通过 Controller 提交编辑结果和操作请求。

2) Toolbar 类

继承 javafx.scene.layout.HBox, 提供图标按钮工具栏。使用 Ikonli 图标库渲染 MaterialDesign2 图标。所有按钮的点击事件直接委托给 MindMapController 的对应方法。

类间关系：持有 MindMapController 引用（关联关系），通过构造器注入。

3) StructureTreeView 类

继承 javafx.scene.layout.BorderPane，使用 JavaFX 的 TreeView<MindMapNode>以树形结构展示节点层次。核心特性是展开状态保持机制——刷新前收集已展开节点的 ID 集合，重建后按 ID 恢复展开状态。

类间关系：持有 MindMapModel 引用（关联关系），通过构造器注入。

4) StatusBar 类

继承 javafx.scene.layout.HBox，显示文件路径、操作描述、节点统计和脏标记。支持临时消息（如“已保存”）的定时显示与自动恢复。

类间关系：独立组件，通过 update()方法接收数据，不持有 Model 或 Controller 引用。

5) WelcomeView 类

继承 javafx.scene.layout.VBox，显示启动欢迎页。通过 Runnable 回调参数接收“新建”和“打开”操作，自身不依赖 Controller。

类间关系：通过回调模式与 MainApp 交互，不直接依赖任何业务类。

2.2.6 工具层

1) FileManager 类

提供静态方法 save(MindMapModel,File)和 load(File):MindMapModel，负责思维导图的序列化与反序列化。使用 Gson 库手动构建 JSON 结构，文件扩展名为.dt。序列化时递归遍历节点树构建 JsonObject；反序列化时通过反射设置节点的 id 字段（因为 id 是 final 字段，无 setter）。

类间关系：所有方法为 static，被 Controller 直接调用（依赖关系）。

2) PreferencesManager 类

提供静态方法 getLastFile()和 setLastFile(String)，读写用户 home 目录下的.mindmap_prefs 配置文件，记录上次打开的文件路径。

类间关系：所有方法为 static，被 MainApp 和 Controller 调用（依赖关系）。

2.3 数据存储的设计

2.3.1 文件导入/导出

项目使用自定义的.dt 文件，本质是 UTF-8 编码的 JSON。在文件进行导出时，流程如下：MindMapModel —递归遍历—> JsonObject —Gson—> .dt 文件。对应的反序列化操作，加载流程为：.dt 文件 —读取字节—> JSON 字符串 —Gson 解析—> JsonObject —递归构建—> MindMapModel。

2.3.2 内存数据结构

数据在内存中以树形结构存储，核心类是 MindMapNode。每个节点同时持有父节点引用和子节点列表，形成双向链表式的树结构。

2.4 界面设计

这个项目的界面设计基于 JavaFX 框架，整体风格模仿 IntelliJ IDEA 的浅色主题。

2.4.1 工具栏

设计要点：

1) 弹性空间实现左右分离：标题在左，按钮组在右，中间用 Region+HBox.setHgrow(spacer,Priority.ALWAYS)填充：

```
Regionspacer=newRegion();
```

```
HBox.setHgrow(spacer,Priority.ALWAYS);//占据所有剩余空间
```

2) 纯图标按钮：所有按钮只显示图标不显示文字，悬停时弹出 Tooltip 提示。使用 Ikonli 图标库的 MaterialDesign2 图标集；

3) 按钮分组：文件操作、撤销/重做、编辑、剪贴板、布局、导出六组，组间用竖向 Separator

分隔。

4) 事件绑定：所有按钮的 `setOnAction` 直接委托给 `MindMapController` 的对应方法，`Toolbar` 本身不包含业务逻辑。

2.4.2 画布区

1) `DrawingPane` 继承 `StackPane`，是界面中最复杂的组件。`StackPane` 的特点是子节点 层叠放置——后添加的在上层。`Canvas` 负责绘制所有节点和连线，`TextArea` 在用户双击节点时弹出用于编辑文本。

2) 在节点坐标设计时分用逻辑坐标和屏幕坐标，确保中心节点逻辑坐标永远为 (0, 0)，布局算法不需要关心平移和缩放。

节点设计简洁美观，选中时有异色提醒效果。绘制节点间曲线时用到两段二次贝塞尔曲线绘制 L 形弯折连线。

3 系统实现

3.1 应用启动流程

`KidxMind` 基于 `JavaFX` 框架构建，应用启动入口为 `MainApp` 类。启动时首先读取用户偏好文件 (`~/mindmap_prefs`) 中记录的上次打开文件路径：若文件存在且可读，则直接加载进入编辑器界面；若不存在，则显示欢迎页供用户选择新建或打开。核心代码：

`@Override`

```
public void start(Stage stage) {
    this.stage = stage;
    MindMapModel model = new MindMapModel();
    controller = new MindMapController(model);
    controller.setStage(stage);

    String lastFile = PreferencesManager.getLastFile();
    if (lastFile != null) {
        controller.loadFromFile(new File(lastFile));
        showEditor();
    } else {
        showWelcome();
    }

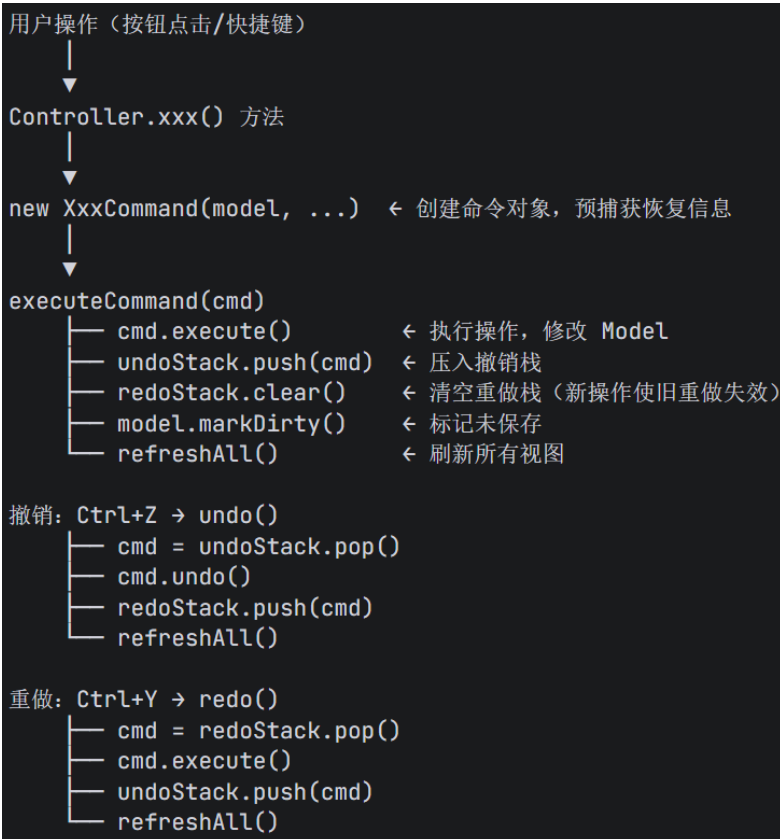
    stage.setTitle("思维导图绘制工具 — KidxMind");
    stage.setMinWidth(800);
    stage.setMinHeight(500);

    stage.setOnCloseRequest(e -> {
        if (!controller.handleExit()) {
            e.consume();
        } else {
            Platform.exit();
        }
    });
}
```

3.2 命令模式与撤销/重做机制

系统采用命令模式实现所有可撤销操作。每个用户操作被封装为一个 `Command` 对象，包含 `execute()` 和 `undo()` 两个方法。`Controller` 维护两个栈：`undoStack` 和 `redoStack`，分别存储已执行和已撤销的命令。

具体流程：



系统共实现了 8 个命令类，覆盖所有可撤销操作。

3.3 布局算法实现

布局计算是思维导图绘制的核心，位于 `DrawingPane` 类中。布局分为两个阶段：自底向上计算尺寸和自顶向下分配位置。

具体流程：



其中 auto 模式:

给定 n 个有顺序的子节点, 每个子节点有一个权重值。在保持原有顺序的前提下, 枚举所有可能的分割点 k (前 k 个放左侧, 剩余放右侧), 选择使左右两侧总高度差最小的方案。

具体代码:

```
private void layoutAuto(MindMapNode root) {
    root.setX(0);
    root.setY(0);
    List<MindMapNode> children = root.getChildren();
    int n = children.size();
    if (n == 0) return;

    // 计算每个子树的权重 (高度)
    double[] weights = new double[n];
    double totalWeight = 0;
    for (int i = 0; i < n; i++) {
        weights[i] = calcSubtreeHeight(children.get(i));
        totalWeight += weights[i];
    }

    // 前缀和: prefixSum[i] = weights[0] + ... + weights[i-1]
    double[] prefixSum = new double[n + 1];
    for (int i = 0; i < n; i++) {
        prefixSum[i + 1] = prefixSum[i] + weights[i];
    }

    // 枚举分割点 k, 找最优
    int bestK = 0;
```

```

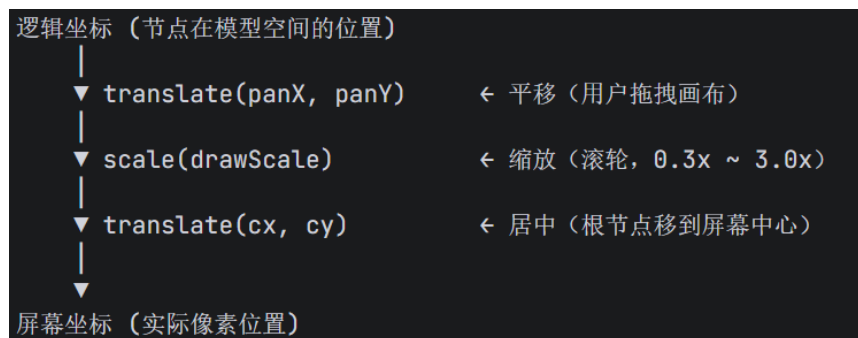
double bestDiff = Double.MAX_VALUE;
for (int k = 0; k <= n; k++) {
    double leftWeight = prefixSum[k];
    double rightWeight = totalWeight - prefixSum[k];
    double leftH = leftWeight + (k > 1 ? (k - 1) * V_GAP : 0);
    double rightH = rightWeight + (n - k > 1 ? (n - k - 1) * V_GAP : 0);
    double diff = Math.abs(leftH - rightH);
    if (diff < bestDiff) {
        bestDiff = diff;
        bestK = k;
    }
}

// 按最佳分割点排列左右两侧
// 左侧组: children[0..bestK-1], 从上到下排列
// 右侧组: children[bestK..n-1], 从上到下排列
// ... (排列代码)
}

```

3.4 画布绘制、坐标变换

绘制采用 JavaFX Canvas, 通过三层坐标变换将逻辑坐标映射到屏幕坐标:



具体代码:

```

private void drawAll() {
    GraphicsContext gc = canvas.getGraphicsContext2D();
    gc.clearRect(0, 0, canvas.getWidth(), canvas.getHeight());

    gc.save();
    // 三层坐标变换
    gc.translate(cx, cy);
    gc.scale(drawScale, drawScale);
    gc.translate(panX, panY);

    // 先绘制所有连线
    drawAllConnections(gc, root, drawScale, cx, cy, panX, panY);
    // 再绘制所有节点 (节点覆盖在连线上方)
    drawAllNodes(gc, root, drawScale, cx, cy, panX, panY);
}

```

```

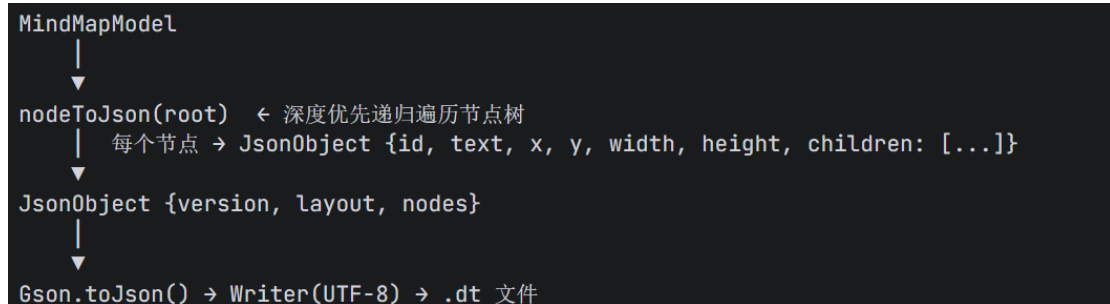
        gc.restore();
    }
}

```

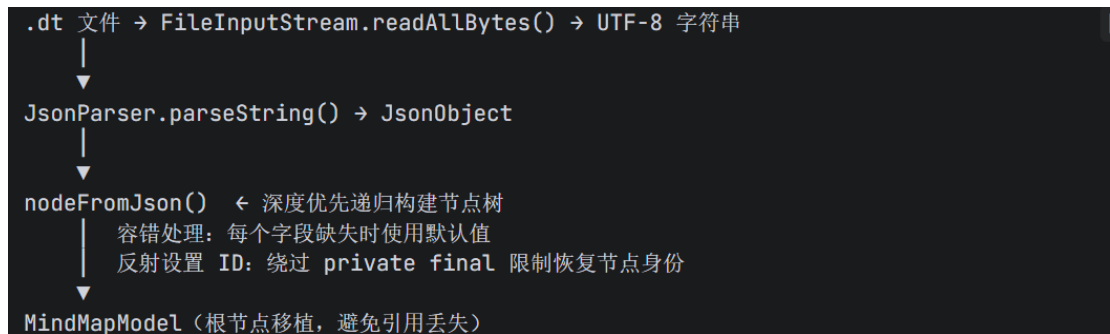
3.5 文件读写实现

文件读写由 `FileManager` 类负责，使用 `Gson` 库进行 JSON 序列化与反序列化。文件扩展名为 `.dt`，本质是 UTF-8 编码的 JSON 文本。

其中保存流程：



加载流程：



核心代码：

序列化：

```

private static JsonObject nodeToJson(MindMapNode node) {
    JsonObject obj = new JsonObject();
    obj.addProperty("id", node.getId());
    obj.addProperty("text", node.getText());
    obj.addProperty("x", node.getX());
    obj.addProperty("y", node.getY());
    obj.addProperty("width", node.getWidth());
    obj.addProperty("height", node.getHeight());
    if (node.getUserWidth() > 0) {
        obj.addProperty("userWidth", node.getUserWidth());
    }
    JsonArray children = new JsonArray();
    for (MindMapNode child : node.getChildren()) {
        children.add(nodeToJson(child));
    }
    obj.add("children", children);
    return obj;
}

```

反序列化:

```
private static MindMapNode nodeFromJson(JsonObject obj) {
    if (obj == null || !obj.has("id")) return null;
    String text = obj.has("text") ? obj.get("text").getString() : "";
    MindMapNode node = new MindMapNode(text);
    setId(node, obj.get("id").getString()); // 反射恢复 ID
    if (obj.has("x")) node.setX(obj.get("x").getAsDouble());
    if (obj.has("y")) node.setY(obj.get("y").getAsDouble());
    // ... 其他字段同理
    if (obj.has("children")) {
        JSONArray children = obj.getAsJSONArray("children");
        for (JsonElement e : children) {
            MindMapNode child = nodeFromJson(e.getAsJsonObject());
            if (child != null) node.addChild(child);
        }
    }
    return node;
}
```

3.6 用户交互实现

系统支持丰富的交互方式，所有交互事件在 `DrawingPane` 中处理。

交互方式	实现方法	处理逻辑
单击节点	<code>handleMousePressed</code>	命中测试 → <code>controller.selectNode()</code>
双击节点	<code>handleMouseClicked</code>	命中测试 → <code>drawingPane.startEdit()</code>
拖拽交换节点	<code>handleMouseDragged</code>	检测拖拽距离 > 阈值 → <code>controller.swapNodes()</code>
拖拽调整宽度	<code>handleMouseDragged</code>	检测鼠标在右边缘 → 实时调整 → <code>controller.resizeNode()</code>
滚轮缩放	<code>handleScroll</code>	以鼠标位置为中心缩放，范围 0.3x ~ 3.0x
右键拖拽平移	<code>handleMouseDragged</code>	更新 <code>panX / panY</code> → <code>redraw()</code>
方向键导航	MainApp 全局事件	<code>controller.navigateUp / Down / Left / Right()</code>

4 系统测试

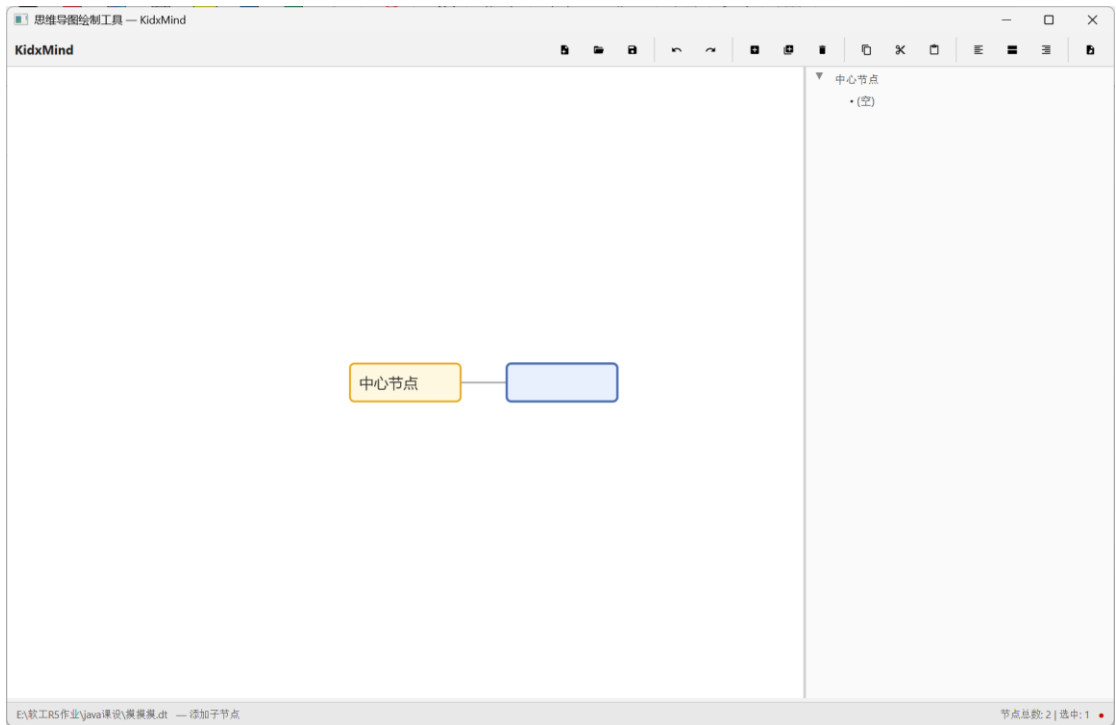
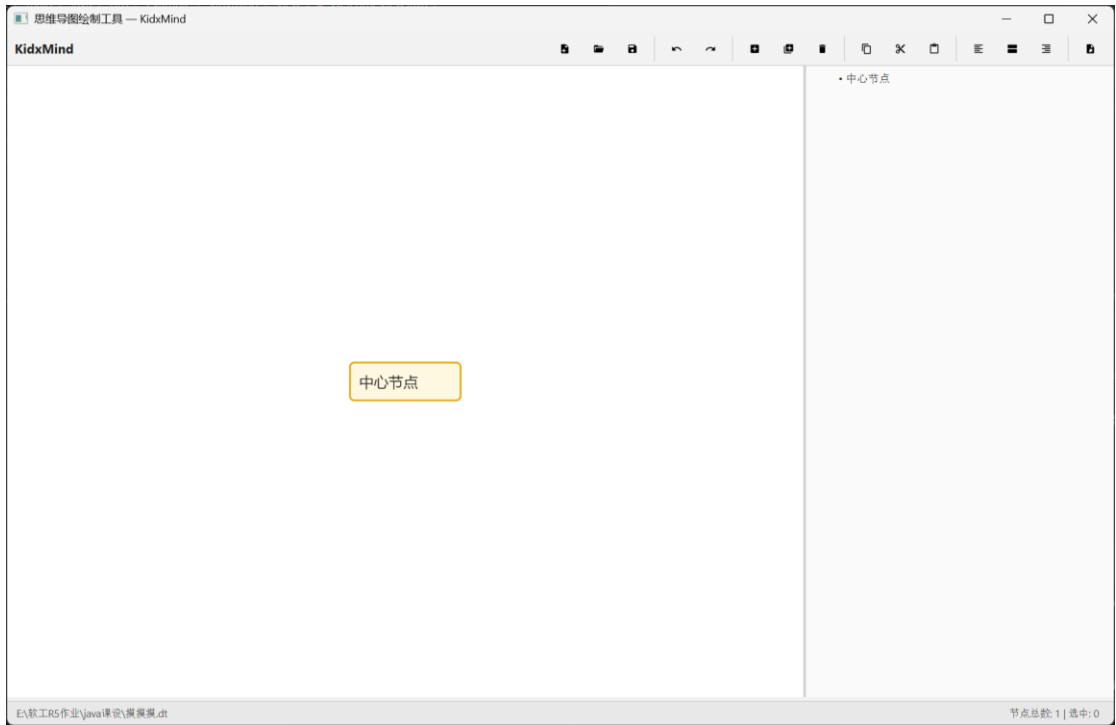
4.1 模块测试

4.1.1 节点添加功能

在思维导图软件中新建空白思维导图后，通过以下步骤验证核心功能：

1. 选中中心节点（默认显示为“中心节点”）
2. 点击新增子节点按钮
3. 观察到成功生成子节点（如图 2 所示，中心节点右侧出现蓝色子节点）

测试结果表明：软件的新增子节点功能正常，能够正确响应操作并生成相应的节点元素，功能完整。另外兄弟节点的添加不多赘述，其功能相似且完整。

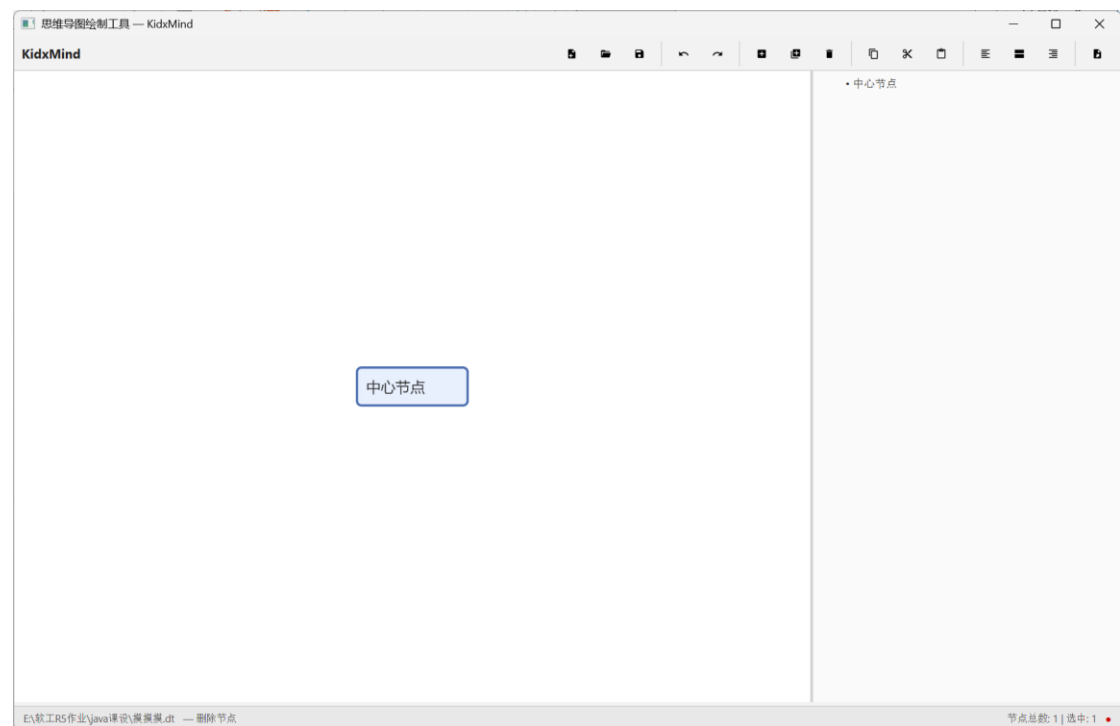
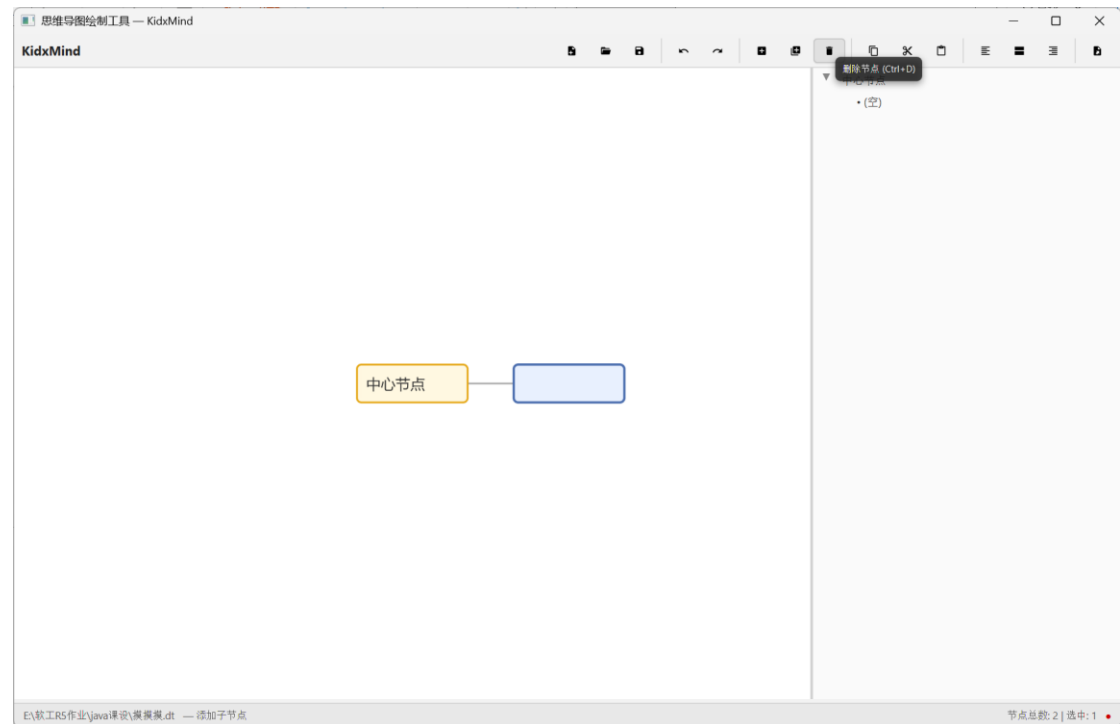


4.1.2 节点删除与撤销

依前实验，进行以下步骤：

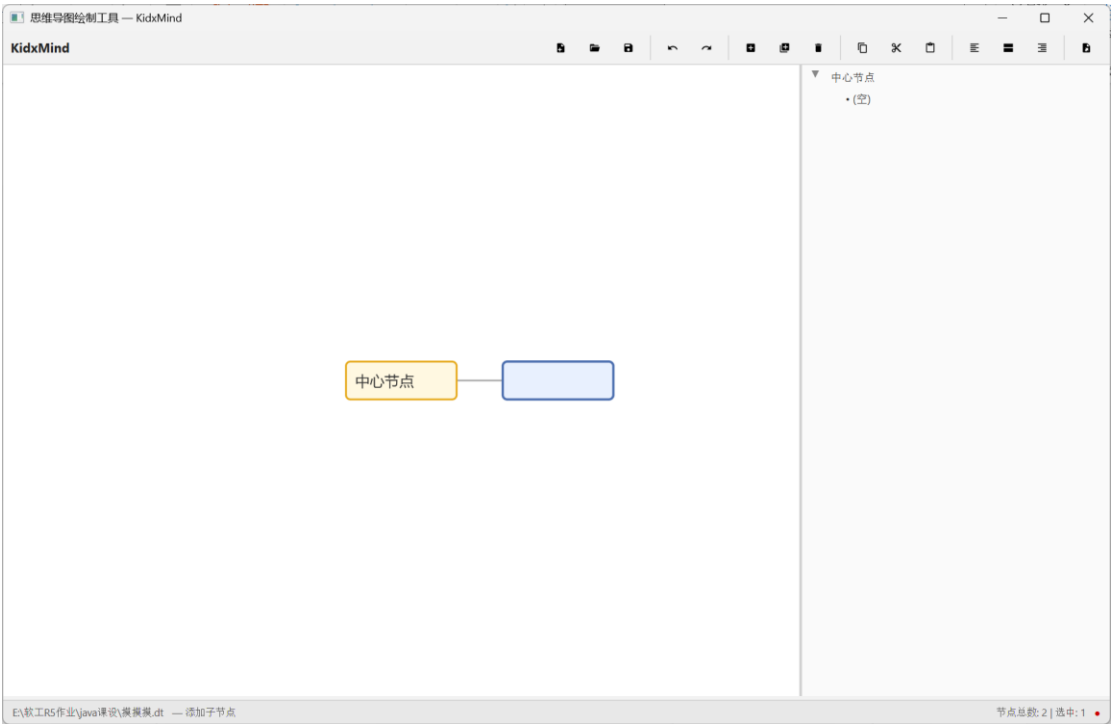
1. 选中子节点（蓝色为已选中）
2. 点击删除子节点按钮
3. 观察到子节点已被删除，默认选择器父节点，下方提示栏显示正确。
4. 再点击撤销按钮。
5. 观察到原先被删除的子节点被恢复。

表明：删除与撤销功能完整。



4.1.3 文本编辑

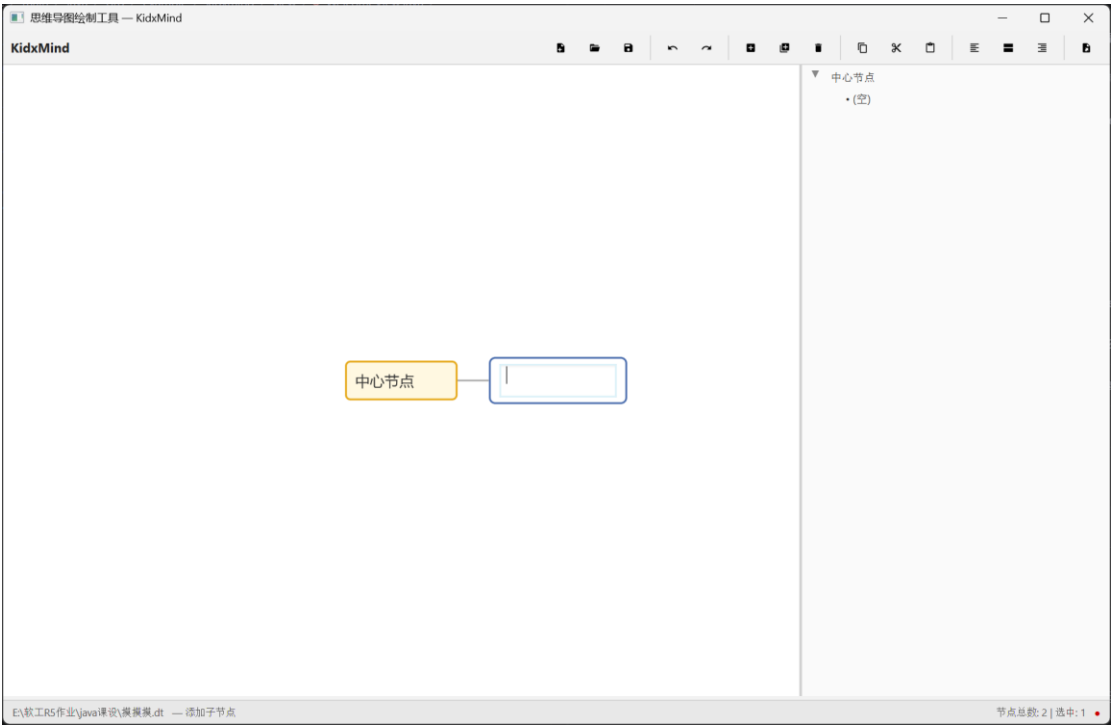
依前实验，新生成一个子节点。

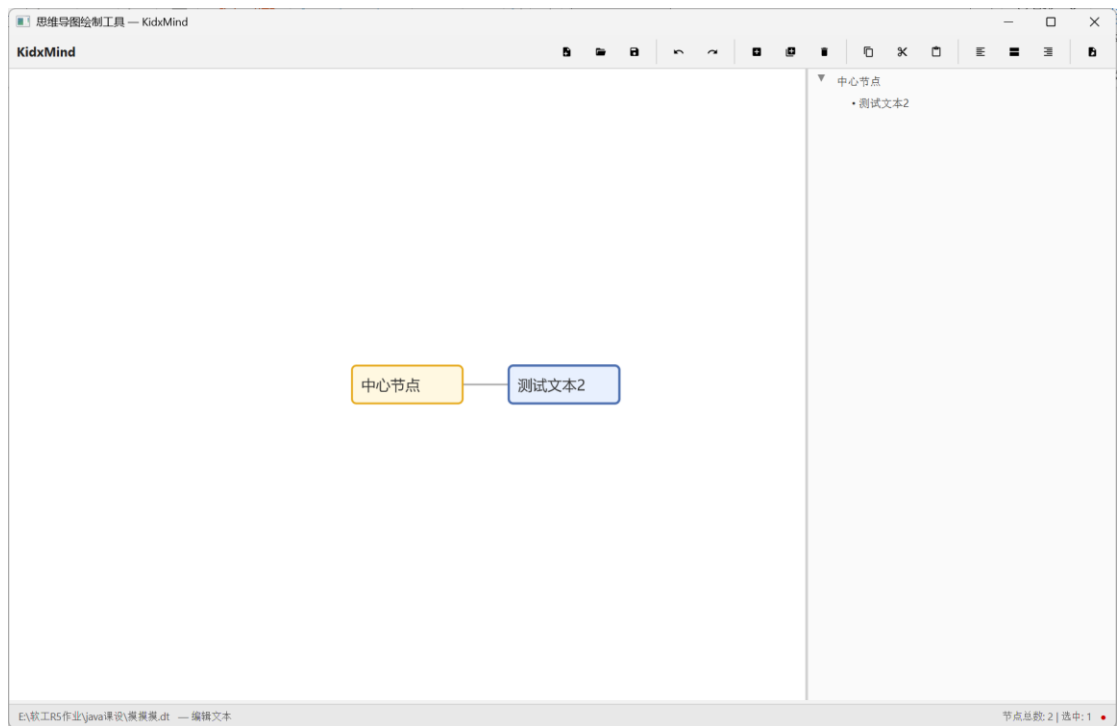
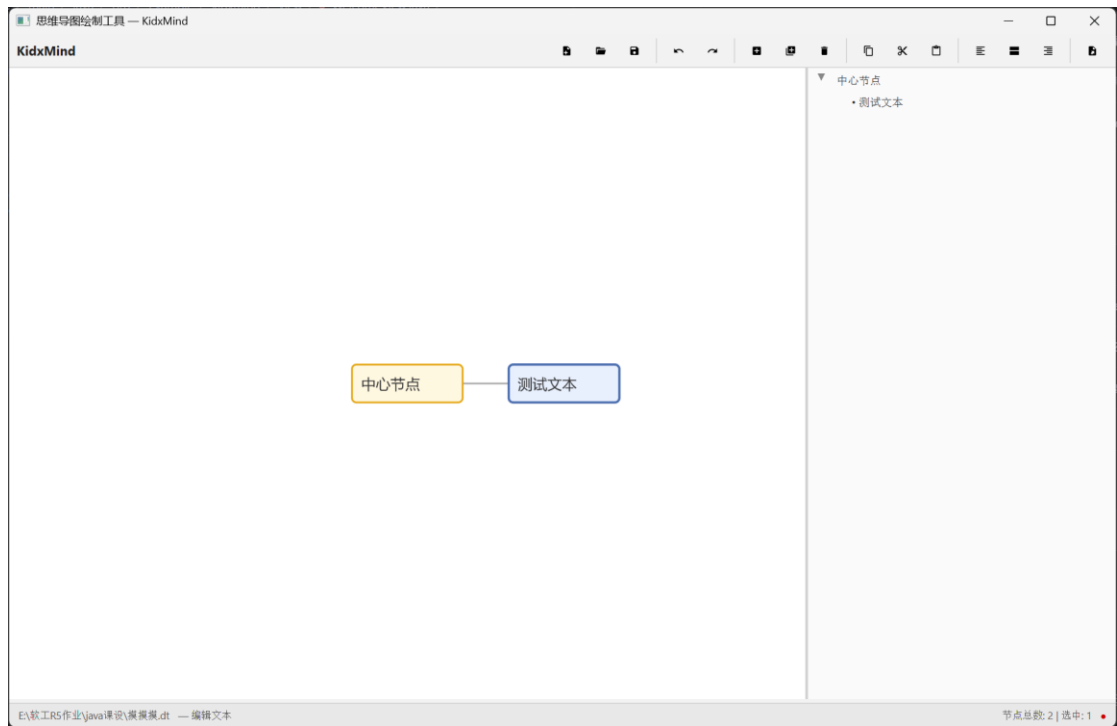


以此进行以下步骤：

1. 双击子节点以进入文本编辑，输入“测试文本”。
2. 观察到字样显现在子节点中间。
3. 再进行双击，修改文本。
4. 观察到文本被修改。

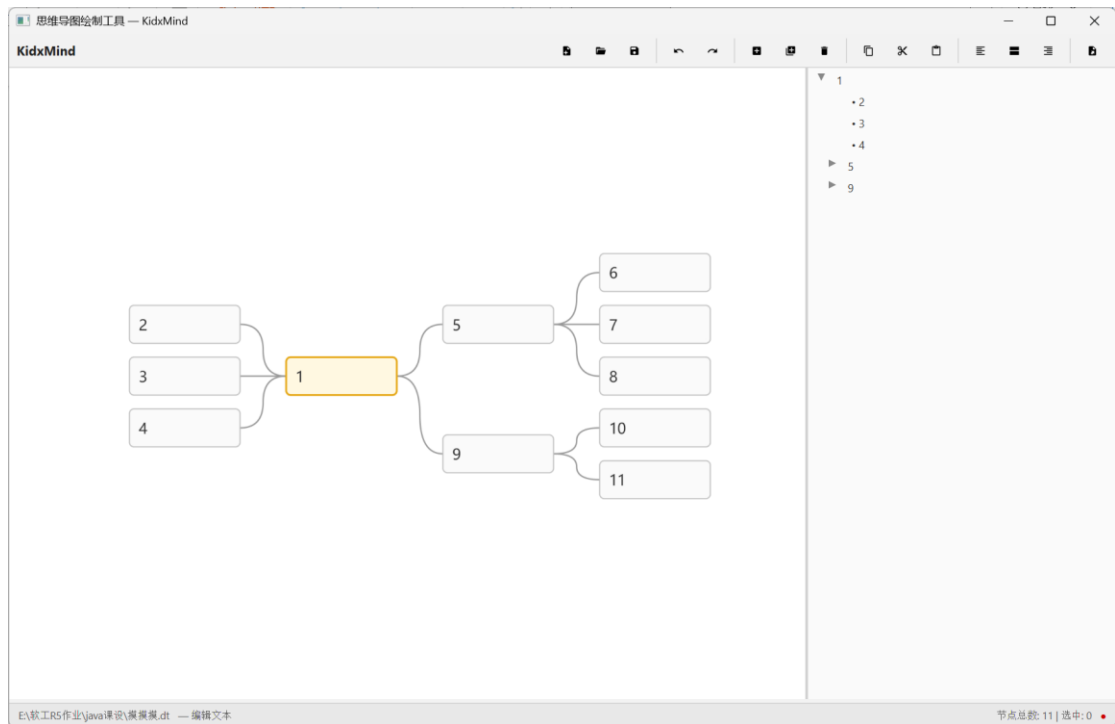
结果表明：文本添加与编辑功能完整。





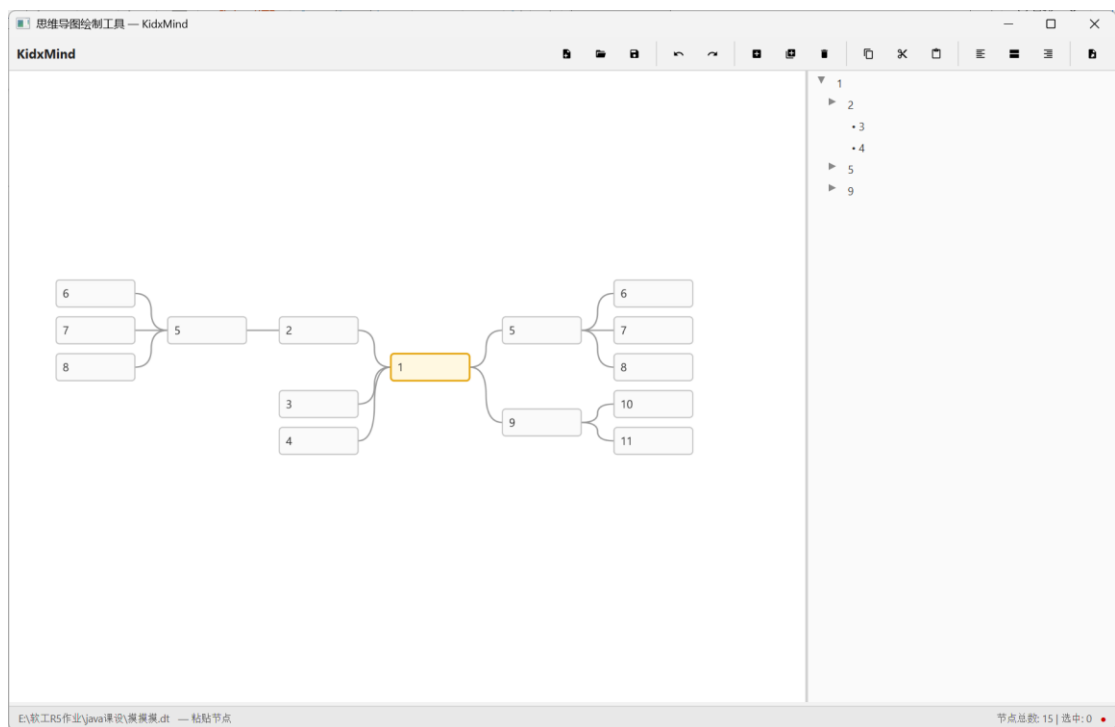
4. 1. 4 剪贴板操作

创建一个多级思维导图。



进行以下实验步骤：

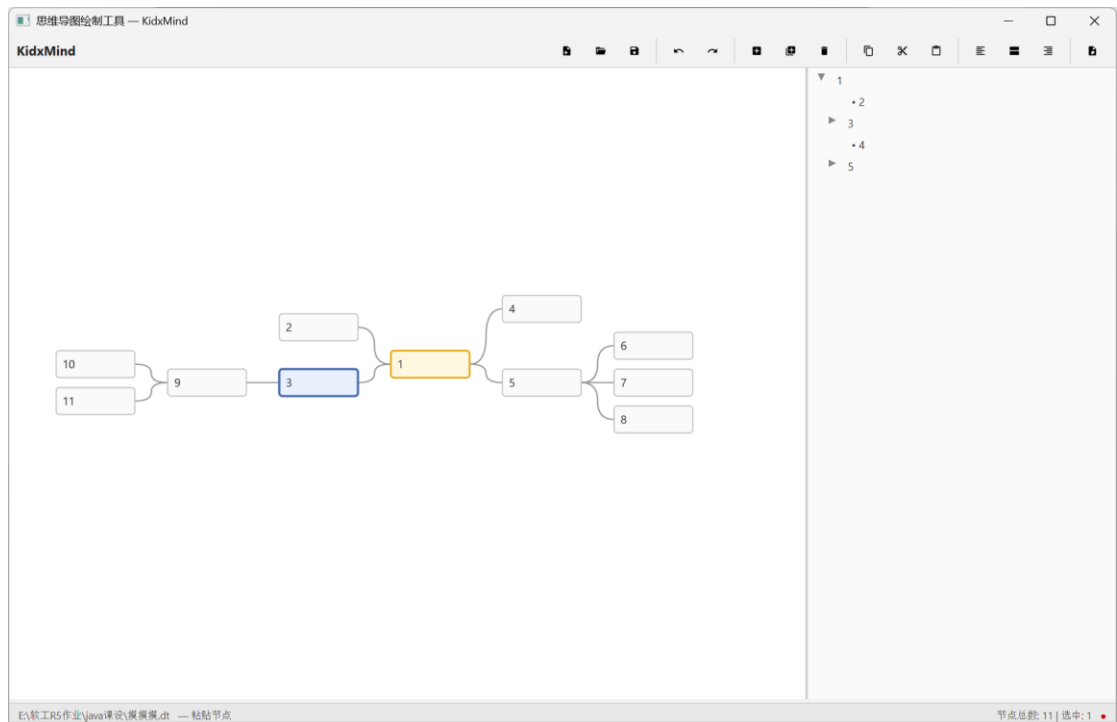
1. 选中节点“5”，点击复制键；选中节点“2”，点击粘贴。
2. 观察到节点“5”及其子节点群被复制到节点“2”后，成为节点“2”的子树。



运用撤回恢复实验前状态。

进行以下实验步骤：

1. 选中节点“9”，点击剪切按钮；选中节点“3”，点击粘贴按钮。
2. 观察到原本节点“9”及其子节点被移动至节点“3”后。



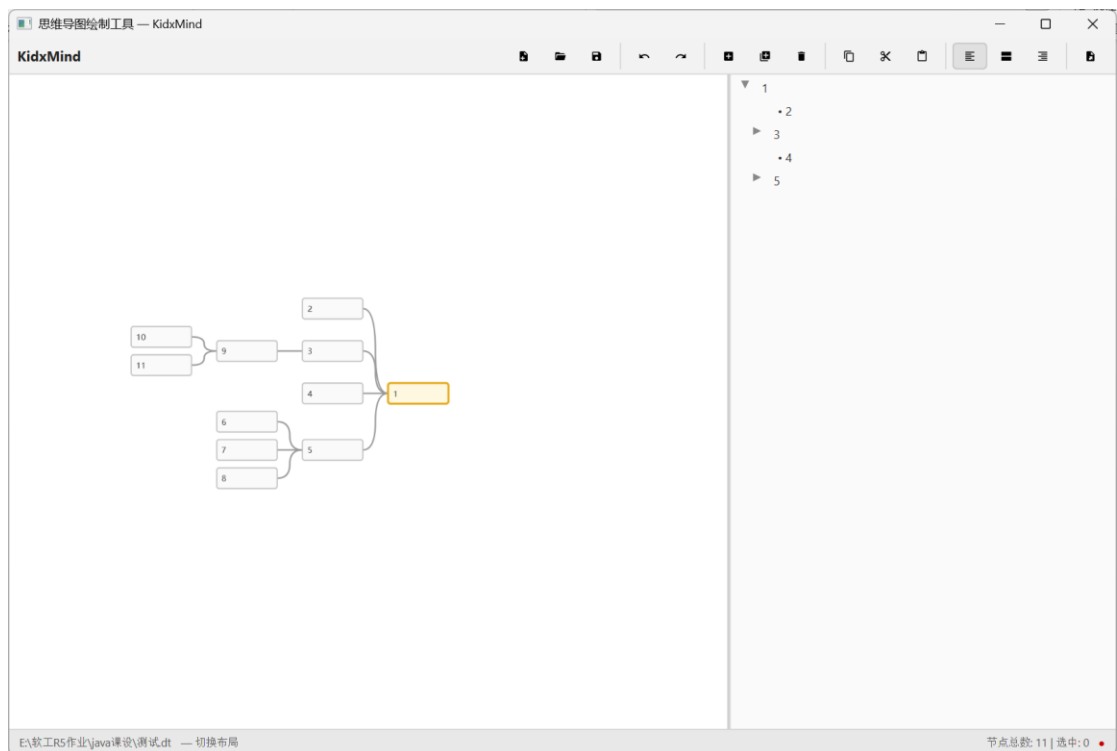
4.1.5 布局模式切换

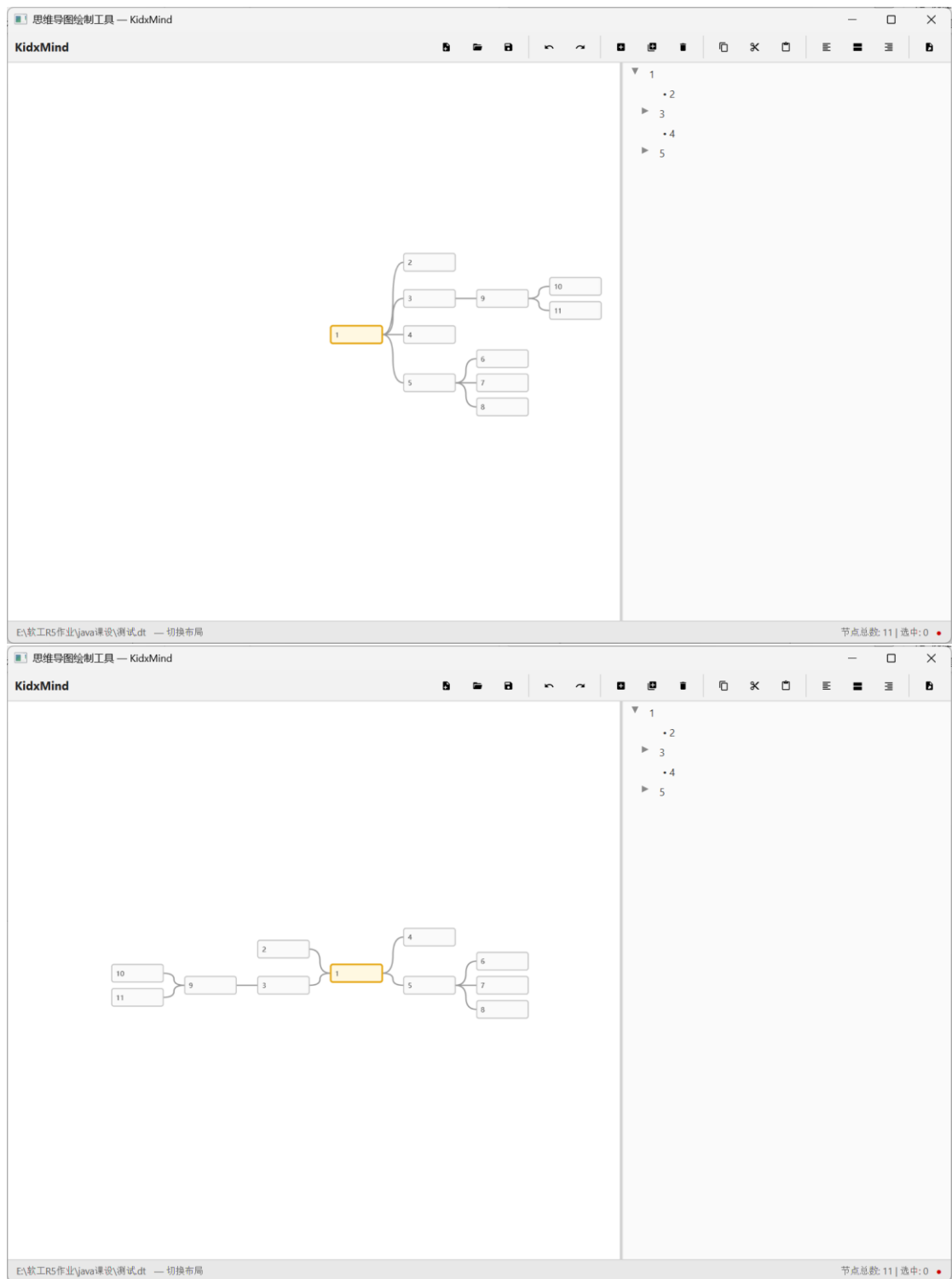
界面默认为 auto 模式。

依前实验，进行以下步骤：

1. 切换至 left 模式；切换至 right 模式；切换至 auto 模式。
2. 观察到界面变化。

结果表明：功能完整，子节点位置重新计算。



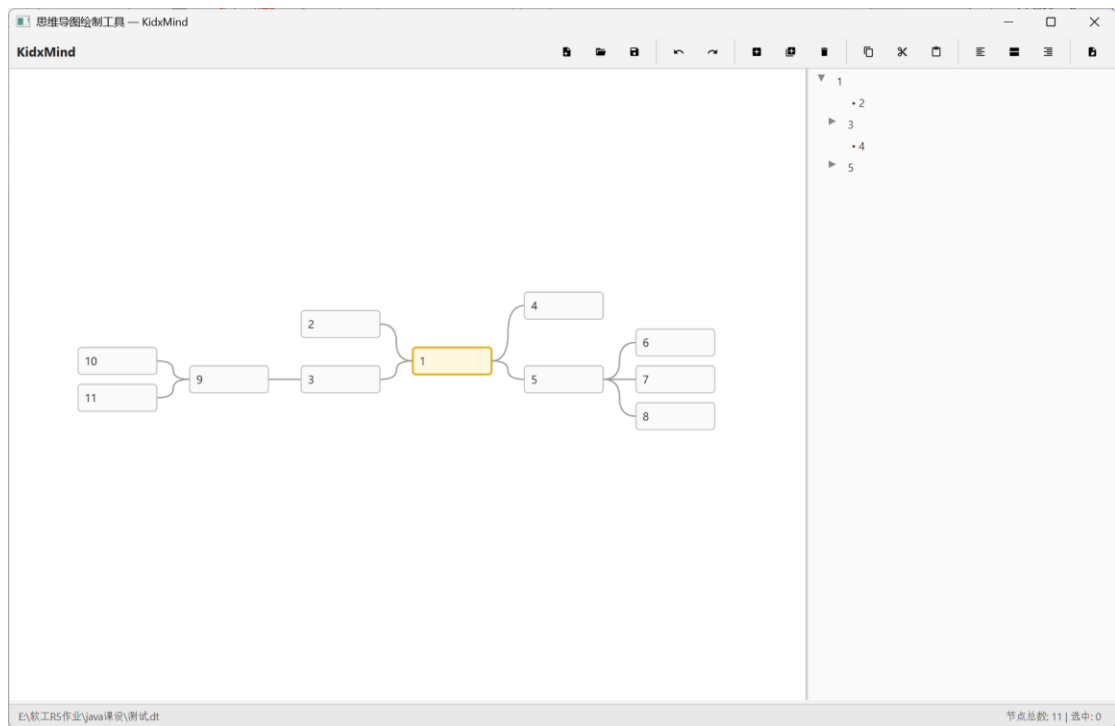


4. 1. 6 文件保存与加载

依前实验，将此文件保存为“测试.dt”。



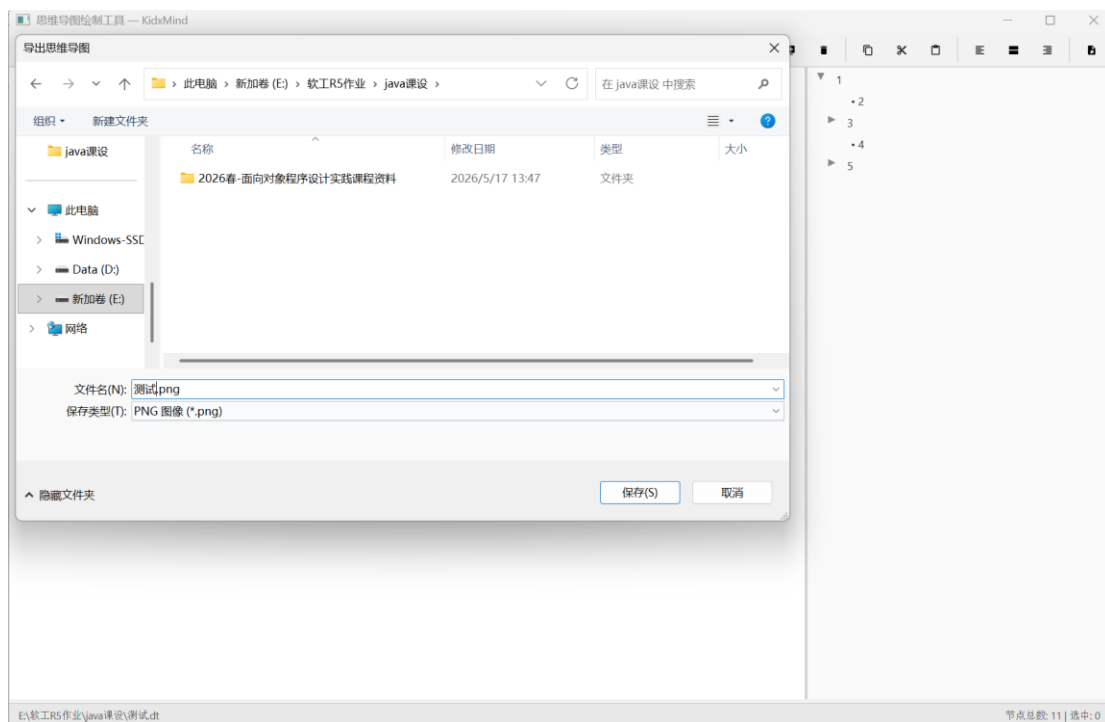
保存成功。再次打开后观察到各子节点位置与文本无变化。



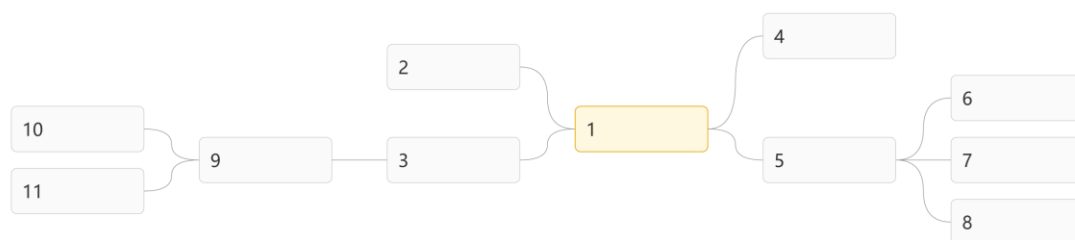
表明功能正常。

4. 1. 7 导出图片

依前实验，点击导出图片按钮，保存至同样位置。



打开图片，发现与原先导图布局模式、节点名字与位置无异。（此为保存图片）



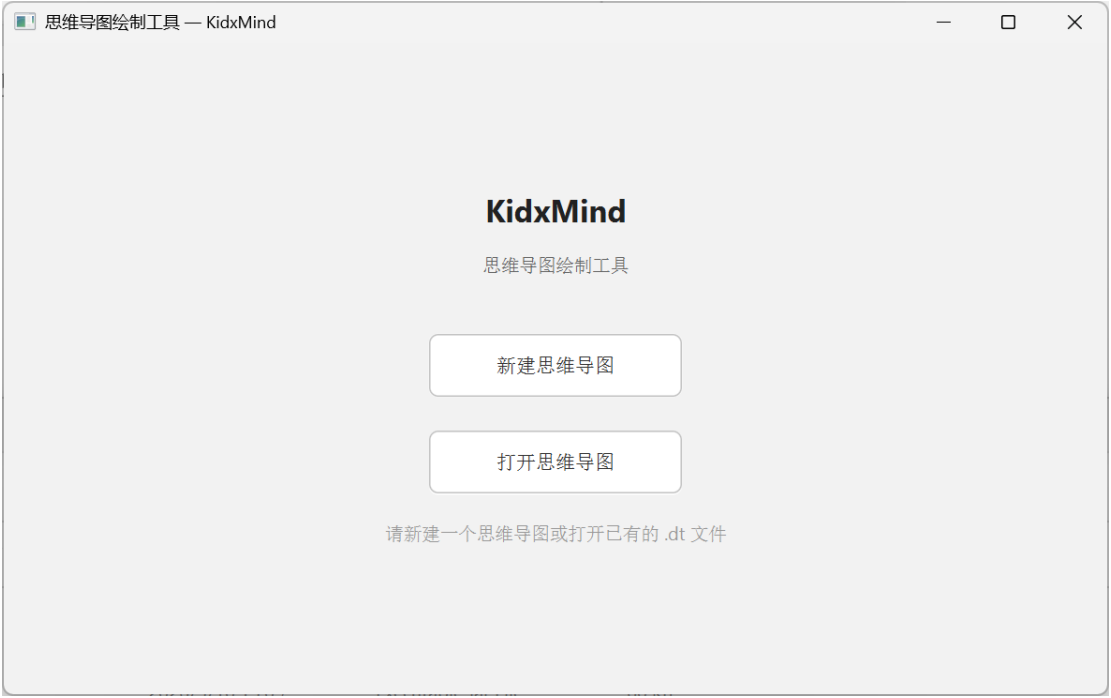
表明：导出图片功能完整。

5 系统界面

5.1 欢迎界面

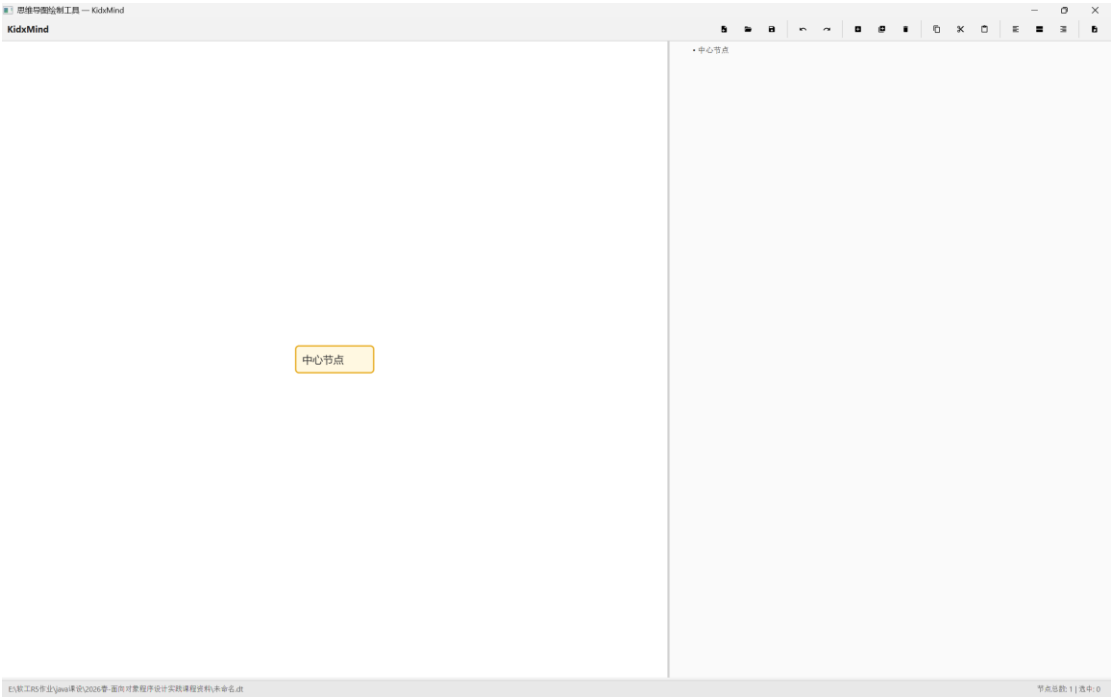
当上次结束时保存的文件仍存在且路径未改变时，直接打开软件将会通过源路径开启上一次

编辑的导图文件，否则显示欢迎界面。



5.2 主编辑器界面

新建文件后，中心显示一个中心节点。



6 总结

成员名称：韦成东

负责内容：软件开发，论文撰写

工作描述：

- 1. 原型软件开发，构建了包含核心功能的初步版本。
- 2. 进行软件测试，按照测试计划执行功能测试、界面测试和兼容性测试。

3. 论文撰写，负责论文的整体结构规划、内容撰写与修改，确保符合学术规范和格式要求。

体会：

通过本次项目，我深入了解了软件开发的完整流程，从原型设计到功能实现，再到测试验证，每个环节都需要严谨的态度和专业的技能。在论文撰写过程中，我学会了如何将技术实践转化为学术表达，提升了逻辑思维和文档编写能力。团队协作让我认识到沟通的重要性，不同角色的配合是项目成功的关键。

成员名称：阮贵鹏

负责内容：全栈开发（前端 UI 设计与后端核心逻辑实现）

工作描述：

1. 负责软件界面设计与实现，构建用户友好的交互界面。
2. 完善核心业务逻辑，优化系统功能实现。

体会：

在项目中，我全面参与了前后端开发工作，深刻体会到全栈开发的挑战与乐趣。UI 设计让我关注用户体验，核心逻辑开发则锻炼了我的编程能力。通过解决实际问题，我提升了问题分析和解决能力。团队协作中，平衡不同需求，这些经验对未来的职业发展大有裨益。